Full Energy-Based Recurrent Transformers for Sequence Prediction

Kenju Tomita · kt7432@g.rit.edu
Rochester Institute of Technology
CSCI 635: Introduction to Machine Learning
Individual Project Proposal · October 9, 2026

Abstract

This project will build and train a full Energy-Based Transformer from scratch. The model will learn an energy function over an observed sequence and a candidate next-byte prediction, then repeatedly update the candidate using energy gradients. Recurrence will reuse the same Transformer parameters across these refinement steps. I will compare this architecture with a standard Transformer and a recurrent prediction-refinement control on language modeling and associative recall. All embeddings, Transformer blocks, and output components will be learned from random initialization. Three-seed experiments will measure prediction quality, calibration, runtime, and memory, including whether additional inference steps help. The goal is to determine whether learned energy minimization provides a useful advantage over direct prediction and ordinary iterative refinement at small model scale.

1. Introduction and Research Objectives

Standard Transformers predict outputs through attention and feedforward computation [1]. Energy-based prediction instead learns a compatibility function between observations and candidate outputs, then uses optimization to refine a candidate. Gladstone et al. develop this approach with Energy-Based Transformers [2]. Earlier work also connects attention, energy functions, and associative memory [3]. These ideas motivate investigating an entire learned energy model, with iterative inference as its prediction mechanism.

The research question is: **Can a full Energy-Based Transformer improve sequence prediction and recall through recurrent energy minimization, and is any improvement worthwhile at equal training or inference cost?** Here, recurrence means repeated refinement of a prediction with shared model weights. The model will use causal attention over the observed prefix; its memory will be measured rather than assumed constant.

Prior results (completed before this course project). Related experiments in my EnergyTransformer project used a different architecture: recurrent energy memory mixed with local attention. At approximately 145,000 parameters, a three-seed CPU study reported mean test loss of **2.5121 bits per byte for the hybrid**, versus **2.5782 for full attention** and **2.5770 for local attention**, at 128-byte context and equal 90-second steady-state update budgets [5]. Compilation and evaluation were excluded. At approximately 10.6 million parameters, a separate historical comparison favored full attention: **1.6633 versus 1.7240 validation bits per byte for pure energy**, after 40.96 million training bytes [6].

A separate small Transformer-energy refiner achieved **76.95% accuracy and 0.7133 negative log-likelihood** on a synthetic linear option task, averaged over three seeds [7]. This tested a decision adapter, not a full language model. These prior findings motivate fresh experiments; they do not establish the proposed EBT's quality, scaling behavior, or general reasoning ability.

2. Methodology and Full Model

Architecture. I will independently implement byte embeddings, positional encoding, multihead attention, normalization, feedforward layers, a candidate embedding, and a scalar energy readout. For a prefix $x_{\leq t}$, candidate logits $z^{(r)}$ define a distribution $p^{(r)}$. Its probability-weighted embedding enters the Transformer alongside observed tokens. The energy readout scores this input/candidate pair:

$$p^{(r)} = \mathrm{softmax}(z^{(r)}),\ E_\theta(x_{\leq t},\ p^{(r)}) \in \mathbb{R}$$

Starting from seeded random candidate logits, prediction is refined recurrently:

$$z^{(r+1)} = z^{(r)} - \eta\, \nabla_{z^{(r)}} E_\theta(x_{\leq t},\ p^{(r)})$$

All model parameters will be trainable. The same energy Transformer is reused at every step; the final candidate distribution produces the next-byte prediction. The architecture will not depend on a frozen encoder, pretrained checkpoint, or an external memory-only energy module. Learned energy is a compatibility signal whose relationship to correctness must be evaluated.

Training and correctness. Following the EBT prediction-training approach [2], I will begin with two refinement steps and average next-byte cross-entropy across refined predictions. Training will differentiate through each energy-gradient update, requiring mixed second derivatives. Detaching candidates between steps will be the initial training policy, with a fully unrolled variant as an optional ablation. PyTorch tensor operations and autograd will support an independent implementation. Explicit attention will be used until double-backward correctness is verified. A serial single-prefix reference will validate the parallel causal mask: each candidate may read only its observed prefix and itself; observed tokens cannot read candidates, and candidates cannot read other predictions or future observations.

Controls and data. Three families will be trained from scratch: (1) a standard causal Transformer with a direct next-byte head; (2) a shared-weight recurrent Transformer that learns direct candidate updates without an energy gradient; and (3) the full EBT. The recurrent control will use the same candidate representation, refinement count, and intermediate prediction supervision, isolating the energy-gradient mechanism.

I will use a 256-byte vocabulary and enwik8, containing Wikipedia text and markup [4], with fixed 90-million/5-million/5-million-byte training/validation/test splits and recorded checksums. The core grid has approximately 150,000 parameters and three seeds per family: nine main fits. A second size near one million parameters is optional. Counts will match within 0.5%; minibatch streams, 128-byte context, effective batch, AdamW, and learning-rate search allowances will be shared. A separate seed will calibrate three learning rates on validation. Profiling will fix the common training-byte budget before main fits; presented and unique bytes will be reported separately.

Hypotheses and ablations. H1: EBT improves held-out prediction loss against both controls at matched parameters/data. H2: extra inference refinement improves EBT quality enough to justify its cost. H3: lower energy tends to identify better candidate predictions on held-out examples. Tests will compare 1/2/4/8 inference steps and separate random initializations. Separate associative-recall episodes will use random key/value pairs, distractors, and queries at varying delays, with split-disjoint episodes and answer-only accuracy. These tasks probe retrieval, not general reasoning.

3. Evaluation, Feasibility, and Timeline

Evaluation. Metrics will include bits per byte, recall accuracy, negative log-likelihood, and Brier score. I will report per-seed outcomes, paired differences, and learning curves. Candidate comparisons within each prefix will test energy-versus-correctness agreement and retain wrong low-energy outputs; decreasing energy alone will not count as better prediction.

The primary comparison matches parameters and training bytes; a separate comparison matches end-to-end training time. Both iterative models will receive the same inference step sweep, with quality plotted against measured latency and memory. Equal steps do not imply equal compute. Profiling includes energy/gradient calls, setup, cache, and peak allocated memory. Fixed nonoverlapping evaluation windows will use a declared reset policy; longer-context results will be labeled extrapolation.

Validation will select learning rate, checkpoint, step size, and inference configuration before testing. Checks cover causality, finite-difference gradients, mixed second derivatives, stability, and restart. Nonfinite training or divergent refinement will stop the affected run, preserving failures.

Feasibility. I will begin with a small single-prefix EBT and synthetic learning check, then implement the batched sequence model and both controls. The planned cap is 12 new GPU hours: two for correctness/profiling, eight for training, and two for evaluation, with a 20% timing margin. If higher-order gradients restrict throughput or GPU access is limited, I will reduce width or context uniformly while retaining the three families and seeds. The study will characterize small-model behavior without foundation-scale extrapolation.

Individual work plan. I will independently implement the architecture and training pipeline, train from random weights, analyze results, credit external resources, and arrange peer proofreading.

Dates	Milestone and output
October 9	Submit the full-EBT proposal and freeze the core research questions.
October 10–16	Implement scalar energy and candidate updates; verify toy gradients; give the proposal presentation.
October 17–November 1	Complete causal sequence EBT, both controls, data pipeline, and trainer; calibrate on validation.
November 2–15	Train the three-seed core grid; run recall experiments and the second size if feasible.
November 16–29	Evaluate refinement/cost tradeoffs, time-matched controls, and calibration; freeze selections and test.
November 30–December 8	Analyze results, prepare figures, obtain peer feedback, and finalize reproducible artifacts.
December 9	Submit the final report and required code/artifacts.
December 10–13	Rehearse and deliver the final presentation.

Expected contribution. Deliverables will be a complete EBT implementation, comparable trained controls, reproducible logs/checkpoints, and quality-versus-cost analysis. The study will establish where recurrent energy optimization helps, where it fails, and whether its additional computational expense is justified.

References

1. Vaswani, A., et al. “Attention Is All You Need.” *NeurIPS*, 2017. Conference proceedings.

2. Gladstone, A., et al. “Energy-Based Transformers Are Scalable Learners and Thinkers.” *ICLR*, 2026. Conference proceedings.

3. Hoover, B., et al. “Energy Transformer.” *NeurIPS*, 2023. Conference proceedings.

4. Mahoney, M. “About the Test Data,” Large Text Compression Benchmark. Dataset source and checksums.

5. EnergyTransformer project. “Compute-Controlled Confirmation.” Prior local exploratory report, September 2026. Local report (included in project repository).

6. EnergyTransformer project. “Reconciled Evidence and Repair Results.” Prior local historical-results audit, September 2026. Local report (included in project repository).

7. EnergyTransformer project. “Calibration and Reproducibility Follow-up.” Prior synthetic decision-adapter study, September 2026. Local report (included in project repository).